



REFACTORIZACION

Subtítulo del documento a
varias líneas





Memoria de Refactorización: Sistema de Reservas de Cine

A continuación, se describen los cambios estructurales y lógicos aplicados al código original para mejorar su **mantenibilidad, legibilidad y robustez**.

1. Extracción del método de inicialización de salas

Se creó el método `inicializarSalas()`. Originalmente, la creación del array y el bucle de instanciación estaban mezclados en el `main`. Ahora el inicio del programa es más limpio.

2. Extracción de la lógica de carga de cartelera

Se implementó `inicializarCartelera()`. Al sacar los datos de prueba del `main`, el código queda preparado para que en el futuro los datos se carguen desde una base de datos o un archivo externo sin modificar la estructura principal.

3. Modularización del Menú Principal

Se extrajo la impresión del menú al método `mostrarMenuPrincipal()`. Esto permite cambiar el diseño visual del menú en un solo lugar sin riesgo de borrar lógica importante del programa.



4. Implementación de lectura de enteros segura

Se creó el método `leerEntero()`. Antes, si el usuario introducía una letra, el programa lanzaba una excepción y se cerraba. Ahora, el método valida la entrada y pide el dato de nuevo hasta que sea correcto.

5. Encapsulamiento de la lógica de reserva

Se extrajo el bloque del "Caso 2" al método `procesoDeReserva()`. Esto redujo el tamaño del `main` drásticamente, facilitando la detección de errores en el flujo de compra.

6. Ajuste de Interfaz de Usuario (UX) de 0-4 a 1-5

Se refactorizó la interacción con el usuario para que use números naturales (1 a 5). El programa resta `-1` internamente para manejar los índices del array, eliminando la confusión para usuarios que no son programadores.

7. Uso de formato profesional con `printf`

En la clase `Cartelera`, se cambió la concatenación manual de strings por `System.out.printf`. Esto garantiza que las columnas de la cartelera (título, sala, hora) queden perfectamente alineadas en la consola.



8. Implementación de reserva múltiple (Bucle anidado)

Se añadió un bucle `do-while` dentro del proceso de reserva. Esto permite al usuario comprar varias entradas para la misma película sin tener que volver a elegir la película y el horario desde cero.

9. Eliminación de variables temporales innecesarias

Se simplificaron expresiones lógicas para que el código fuera más directo, reduciendo el uso de memoria y mejorando la velocidad de lectura del código.

10. Mejora del encapsulamiento de atributos

Se cambiaron los modificadores de acceso de los atributos de las clases a `private` y se crearon los métodos `Getter` correspondientes, siguiendo las reglas de oro de la Programación Orientada a Objetos.

11. Refactorización del dibujo de la sala

Se utilizó un **operador ternario** en el método `mostrarAsientos()` de la clase `Sala` para simplificar la lógica de impresión de los corchetes `[ X ]` o `[ fila,col ]`.

12. Claridad en nombres de variables



Se renombraron variables genéricas (como `p`, `s`, `c`) por nombres descriptivos (`pelicula`, `sala`, `cartelera`), cumpliendo con los estándares de *Clean Code*.

13. Agrupación de validaciones de rango

Se unificaron las comprobaciones de que la fila y columna estén dentro del rango (1-5) en una sola estructura condicional, evitando anidamientos excesivos (el llamado "código espagueti").

14. Cierre controlado de recursos

Se añadió el cierre del objeto `Scanner` (`sc.close()`) al final del `main`, asegurando que no queden fugas de memoria (memory leaks) al terminar la ejecución del programa.